IELTS GT WRITING EVALUATION PLATFORM

Complete Backend Architecture & System Design

Trainer-First B2B SaaS • Multi-Tenant • AI-Powered

1. System Overview

This document defines the complete implementable logic for a production-ready, multi-tenant SaaS platform that enables IELTS coaching institutes to assign writing tasks, collect student submissions, evaluate them using an AI engine aligned to official IELTS band descriptors, and surface analytics to trainers.

> **Design Philosophy**
> Every module is stateless-first, event-driven where appropriate, and LLM-calls are treated as expensive I/O — cached, normalised, and rate-limited. No UI is defined. All logic is expressed in data models, API contracts, and pseudocode.

1.1 High-Level Flow

- Trainer creates organisation → invites students → generates/assigns prompts
- Student submits writing response against an assigned prompt
- Evaluation pipeline triggers: rule-based pre-check → LLM scoring → score normalisation → feedback generation
- Trainer reviews AI evaluation, optionally overrides → approves
- Analytics engine aggregates results → surfaces progression, weakness patterns, predictions

2. Architecture Diagram (Textual)

```
┌──────────────────────────────────────────────────────────────────────┐
│                          API GATEWAY                                 │
│              (Auth middleware, rate limiting, routing)               │
└──────────────────────────────────────────────────────────────────────┘
            │                    │                    │
            ▼                    ▼                    ▼
    ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐
    │  Prompt Svc  │   │ Submission Svc │  │  Analytics Svc   │
    └──────────────┘   └──────────────┘   └──────────────────┘
            │                    │                    │
            │            ┌──────────────┐             │
            │            │  Evaluation Svc  │──────────┤
            │            │  (AI Pipeline)   │          │
            │            └──────────────┘             │
            │                    │                    │
    ┌──────────────────────────────────────────────────────────┐
    │                  PostgreSQL (Primary DB)                 │
    │   + Redis (cache/queue) + S3 (file storage)              │
    └──────────────────────────────────────────────────────────┘

External: Anthropic Claude API  |  OpenAI Whisper (optional)
```

> **Tech Stack Recommendation**

Backend: Python (FastAPI) or Node.js (Fastify) | DB: PostgreSQL + pgvector | Cache: Redis | Queue: BullMQ or Celery | Storage: S3/R2 | LLM: Claude claude-sonnet-4-20250514 via Anthropic SDK

3. Data Models (DB Schema)

All tables use UUID primary keys, created_at/updated_at timestamps (auto-managed), and soft-delete via deleted_at. Row-level security enforced by org_id on every tenant-scoped table.

3.1 organizations

Column	Type	Notes
id	UUID PK	
name	VARCHAR(255)	Institute name
slug	VARCHAR(100) UNIQUE	URL-safe identifier
plan	ENUM(free,pro,enterprise)	Billing tier
max students	INT	Plan limit
ai tokens used	BIGINT DEFAULT 0	Monthly usage tracking
settings	JSONB	Feature flags, branding
created at	TIMESTAMPTZ	Auto

3.2 users

Column	Type	Notes
id	UUID PK	
org_id	UUID FK → organizations	
email	VARCHAR(255) UNIQUE	
password hash	VARCHAR	bcrypt
role	ENUM(admin,trainer,student)	
name	VARCHAR(255)	
target band	DECIMAL(2,1)	Student only — e.g. 7.0
metadata	JSONB	Flexible attributes

3.3 prompts

Column	Type	Notes
id	UUID PK	
org_id	UUID FK	NULL = global/system prompt
created_by	UUID FK → users	

Column	Type	Notes
task_type	ENUM(task1_formal, task1_semiformal, task1_informal, task2_opinion, task2_discussion, task2_problem_solution, task2_adv_disadv)	
difficulty	DECIMAL(2,1)	Maps to target band 4.0–9.0
topic_tags	TEXT[]	e.g. ['work', 'health']
body	TEXT	Full prompt text
word_limit_min	INT	150 or 250
is_ai_generated	BOOLEAN DEFAULT false	
status	ENUM(draft,active,archived)	

3.4 assignments

Column	Type	Notes
id	UUID PK	
org_id	UUID FK	
prompt_id	UUID FK → prompts	
assigned_by	UUID FK → users (trainer)	
assignee_type	ENUM(student,group)	
assignee_id	UUID	student_id or group_id
due_at	TIMESTAMPTZ	Deadline
allow_resubmit	BOOLEAN DEFAULT true	
max_attempts	INT DEFAULT 3	
status	ENUM(pending,active,closed)	

3.5 submissions

Column	Type	Notes
id	UUID PK	
assignment_id	UUID FK → assignments	
student_id	UUID FK → users	
attempt_number	INT DEFAULT 1	
body	TEXT	Full submission text
word_count	INT	Pre-computed on insert
time_taken_seconds	INT	Client-reported
is_final	BOOLEAN DEFAULT false	Drafts = false
submitted_at	TIMESTAMPTZ	NULL until finalised

Column	Type	Notes
is_late	BOOLEAN DEFAULT false	Computed vs due_at

3.6 evaluations

Column	Type	Notes
id	UUID PK	
submission_id	UUID FK → submissions	
version	INT DEFAULT 1	Increments on override
source	ENUM(ai,trainer,hybrid)	
status	ENUM(pending,processing,completed,failed,approved)	
band_ta	DECIMAL(2,1)	Task Achievement
band_cc	DECIMAL(2,1)	Coherence & Cohesion
band_lr	DECIMAL(2,1)	Lexical Resource
band_gra	DECIMAL(2,1)	Grammatical Range & Accuracy
band_overall	DECIMAL(2,1)	Computed: avg(ta+cc+lr+gra) rounded to 0.5
feedback_json	JSONB	Full structured feedback (see §5)
model_answer	TEXT	Band 7+ rewrite
llm_model	VARCHAR	e.g. claude-sonnet-4-20250514
llm_tokens_used	INT	Prompt + completion
processing_ms	INT	Latency tracking
approved_by	UUID FK → users	Trainer who approved
approved_at	TIMESTAMPTZ	

3.7 evaluation_overrides

Column	Type	Notes
id	UUID PK	
evaluation_id	UUID FK → evaluations	
trainer_id	UUID FK → users	
field_changed	VARCHAR	e.g. 'band_ta', 'feedback_json'
old_value	JSONB	Snapshot before change
new_value	JSONB	Value after change
reason	TEXT	Trainer justification

Column	Type	Notes
created_at	TIMESTAMPTZ	Audit trail

3.8 error_patterns (per student)

Column	Type	Notes
id	UUID PK	
student_id	UUID FK → users	
pattern_type	ENUM(grammar, vocabulary, coherence, task_response, paragraphing, run_on, intro_weakness, conclusion_weakness)	
occurrence_count	INT DEFAULT 1	Incremented per detection
last_seen_at	TIMESTAMPTZ	
last_submission_id	UUID FK → submissions	
examples	JSONB	Array of {sentence, issue} objects

3.9 analytics_snapshots (materialised daily)

Column	Type	Notes
id	UUID PK	
org_id	UUID FK	
student_id	UUID FK	NULL = class-level
snapshot_date	DATE	
avg_band_ta	DECIMAL(3,2)	
avg_band_cc	DECIMAL(3,2)	
avg_band_lr	DECIMAL(3,2)	
avg_band_gra	DECIMAL(3,2)	
avg_band_overall	DECIMAL(3,2)	
submission_count	INT	
predicted_target_date	DATE	ML estimate

4. Core Modules

4.1 Prompt Generation Module

Handles both AI-generated and manually authored prompts.

AI Generation Flow (Pseudocode)

```
func generate_prompt(task_type, difficulty, topic_tags, org_id):
  system = build_prompt_system_prompt(task_type, difficulty)
  user   = f'Generate an IELTS GT {task_type} prompt on topics: {topic_tags}'
  raw    = llm_call(system, user, max_tokens=400)
  parsed = parse_prompt_json(raw)  # {body, scenario, bullet_points}
  return db.insert('prompts', {...parsed, org_id, is_ai_generated: true})
```

Prompt System Prompt Template

```
'You are an official IELTS examiner. Generate a {task_type} writing prompt
 targeting Band {difficulty} candidates. Output JSON only:
 {"body": "<full task text>", "scenario": "<context>",
  "bullet_points": ["point1","point2","point3"]}
 The prompt must be realistic, unambiguous, and match IELTS GT conventions.'
```

4.2 Assignment Module

Assignment to Individual

```
POST /assignments
{ prompt_id, assignee_type: 'student', assignee_id, due_at, max_attempts }
```

Assignment to Group

```
// Expand group → individual assignment rows on write
func assign_to_group(group_id, prompt_id, due_at):
  students = db.query('SELECT id FROM group_members WHERE group_id = ?', group_id)
  batch_insert('assignments', students.map(s => ({...params, assignee_id: s.id})))
```

Late Submission Detection

```
// Run on submission finalisation
func mark_late(submission):
  assignment = db.get(submission.assignment_id)
  if assignment.due_at AND now() > assignment.due_at:
    db.update('submissions', {is_late: true}, submission.id)
```

4.3 Submission Module

Supports draft saves and final submission with full version history.

```
func save_submission(assignment_id, student_id, body, is_final):
  wc = count_words(body)
  min_words = get_min_words(assignment.task_type)  // 150 or 250
  if is_final AND wc < min_words: raise ValidationError(...)
  attempt = get_attempt_count(assignment_id, student_id) + 1
  sub = db.insert('submissions', {assignment_id, student_id, body,
                  word_count: wc, is_final, attempt_number: attempt})
  if is_final: emit_event('submission.finalised', sub.id)
  return sub
```

5. Evaluation Engine (Critical)

The evaluation engine is the core differentiator. It follows a multi-stage pipeline designed to maximise consistency, reduce hallucination, and produce band scores aligned to official IELTS descriptors.

5.1 Pipeline Overview

- Stage 1 — Pre-Check (Rule-Based): word count validation, off-topic flag, language detection
- Stage 2 — Criteria Scoring (LLM): independent calls per criterion to reduce cross-contamination
- Stage 3 — Score Normalisation: statistical rounding + cross-run averaging
- Stage 4 — Feedback Generation (LLM): paragraph-level and sentence-level feedback
- Stage 5 — Error Pattern Extraction: rule + AI hybrid tagging
- Stage 6 — Model Answer Generation (LLM): Band 7+ rewrite
- Stage 7 — Storage & Event Emit: persist evaluation, trigger analytics update

5.2 Stage 1 — Pre-Check (Rule-Based)

```
func pre_check(submission, prompt):
  // 1. Word count
  if submission.word_count < min_words * 0.75:
    return {pass: false, reason: 'too_short', word_count: submission.word_count}
  // 2. Language detection (langdetect lib)
  if detect_language(submission.body) != 'en':
    return {pass: false, reason: 'non_english'}
  // 3. Gibberish check (avg word length, entropy)
  if entropy_score(submission.body) < THRESHOLD:
    return {pass: false, reason: 'gibberish'}
  return {pass: true}
```

5.3 Stage 2 — Criteria Scoring (LLM)

CRITICAL DESIGN DECISION: Each of the 4 criteria is scored in an isolated LLM call. This prevents the model from anchoring on one score to justify another, and allows independent retries on failure.

Scoring Call Structure (per criterion)

```
func score_criterion(criterion, submission_body, prompt_body, task_type):
  system = CRITERION_SYSTEM_PROMPTS[criterion]
  user = build_scoring_user_prompt(submission_body, prompt_body, task_type)
  raw = llm_call(system, user, max_tokens=600, temperature=0.1)
  return parse_criterion_json(raw)  // {band, evidence, errors[]}
```

Temperature Note

Use temperature=0.1 for scoring calls to maximise consistency. Use temperature=0.7 for feedback narrative and model answer generation to allow natural language variety.

Task Achievement System Prompt

```
'You are a strict IELTS examiner scoring ONLY Task Achievement (TA).
 You must ignore grammar, vocabulary, and cohesion entirely.
 Focus exclusively on: Did the student address all bullet points/requirements?
 Is the tone appropriate (formal/informal)? Is the purpose clear?
 Score on the official IELTS 0-9 band scale (use 0.5 increments).
 Return JSON only: {"band": <float>, "evidence": "<2 sentence justification>",
  "errors": [{"type": "task_response_gap", "detail": "..."}]}'
```

Apply the same pattern for CC, LR, and GRA with their respective descriptors. Each system prompt must open with 'Score ONLY [criterion]. Ignore all other criteria.' to prevent bleed-over.

Overall Band Calculation

```
func compute_overall_band(ta, cc, lr, gra):
  avg = (ta + cc + lr + gra) / 4.0
  // Round to nearest 0.5 per official IELTS rules
  return round(avg * 2) / 2.0
```

5.4 Stage 3 — Score Normalisation

To reduce variance across runs (LLMs are non-deterministic even at low temperature), run each scoring call N=3 times and apply the following normalisation:

```
func normalise_scores(criterion, runs):  // runs = [7.0, 6.5, 7.0]
  // 1. Drop outliers (any score > 1.5 bands from median)
  median = statistics.median(runs)
  valid = [r for r in runs if abs(r - median) <= 1.5]
  if len(valid) < 2: valid = runs  // fallback: use all
  // 2. Average and round to 0.5
  avg = sum(valid) / len(valid)
  return round(avg * 2) / 2.0
```

Cost Note

Running 3x scoring calls per criterion = 12 LLM calls per submission. Mitigate cost by: caching identical (prompt+submission) pairs (hash match), using a cheaper model for runs 2+3 and the main model for run 1 only, or running normalisation only for borderline scores (e.g. where run1 is x.0 or x.5 boundary).

5.5 Stage 4 — Feedback Generation

Paragraph-Level Feedback

```
func generate_paragraph_feedback(submission_body, scores, task_type):
  paragraphs = split_into_paragraphs(submission_body)
  system = FEEDBACK_SYSTEM_PROMPT  // see §8
  user = f'''
    Submission (split into paragraphs):
    {json(paragraphs_with_index)}
    Scores: TA={scores.ta}, CC={scores.cc}, LR={scores.lr}, GRA={scores.gra}
    For each paragraph, provide feedback JSON.
  '''
```

```
  return llm_call(system, user, max_tokens=1000, temperature=0.5)
```

Feedback JSON Schema

```
{
  "paragraph_feedback": [
    {
      "para_index": 0,
      "para_text": "<first 80 chars>...",
      "issues": [
        {"type": "grammar|vocabulary|coherence|task_response",
         "sentence": "<offending sentence>",
         "correction": "<corrected version>",
         "explanation": "<why this is wrong>"}
      ],
      "positive": "<what this paragraph does well>"
    }
  ],
  "why_this_score": "<global 3-sentence explanation of overall band>",
  "how_to_improve": {
    "ta": "<specific actionable advice for TA>",
    "cc": "<specific actionable advice for CC>",
    "lr": "<specific actionable advice for LR>",
    "gra": "<specific actionable advice for GRA>"
  },
  "next_band_tips": ["<tip1>", "<tip2>", "<tip3>"]
}
```

5.6 Stage 5 — Error Pattern Extraction

Hybrid rule-based + AI approach. Rules run first (cheap), AI fills gaps (expensive).

Rule-Based Detectors

```
func detect_rule_patterns(text):
  patterns = []
  // Run-on sentences: sentence > 50 words with no internal punctuation
  for sent in sentences(text):
    if word_count(sent) > 50 and comma_count(sent) == 0:
      patterns.append({type: 'run_on', sentence: sent})
  // Paragraphing: para count < 3 for Task 2
  if task_type == 'task2' and paragraph_count(text) < 3:
      patterns.append({type: 'poor_paragraphing'})
  // Weak intro: first para < 25 words
  if word_count(paragraphs[0]) < 25:
      patterns.append({type: 'intro_weakness'})
  // Weak conclusion: last para < 20 words
  if word_count(paragraphs[-1]) < 20:
      patterns.append({type: 'conclusion_weakness'})
  return patterns
```

Pattern Storage (Upsert per Student)

```
func upsert_error_patterns(student_id, patterns, submission_id):
  for p in patterns:
    existing = db.findOne('error_patterns', {student_id, pattern_type: p.type})
    if existing:
      db.update({occurrence_count: existing.count+1, last_seen_at: now(),
                 last_submission_id: submission_id})
    else:
      db.insert({student_id, pattern_type: p.type, occurrence_count: 1,
                 examples: [p.evidence], last_submission_id: submission_id})
```

5.7 Stage 6 — Model Answer Generation

```
func generate_model_answer(prompt_body, submission_body, task_type):
  system = f'''You are an expert IELTS writer. Rewrite the student submission
    to achieve Band 7.5+. Preserve the student's original ideas and structure
    where possible — do not invent new content. Improve: grammar precision,
    vocabulary range, cohesion devices, and task response completeness.
    Output ONLY the rewritten text, no commentary.'''
  user = f'Original task:\n{prompt_body}\n\nStudent draft:\n{submission_body}'
  return llm_call(system, user, max_tokens=600, temperature=0.7)
```

6. Trainer Override System

6.1 Override Flow

- Trainer fetches evaluation via GET /evaluations/{id}
- Trainer PATCHes specific fields (band scores, feedback sections)
- System creates new evaluation version (version + 1), copies old record, applies delta
- Audit row inserted into evaluation_overrides with field_changed, old_value, new_value, reason
- Trainer calls POST /evaluations/{id}/approve to finalise

6.2 Version Control Logic

```
func override_evaluation(eval_id, trainer_id, changes, reason):
  current = db.get('evaluations', eval_id)
  // Snapshot changed fields into audit table
  for field, new_val in changes:
    db.insert('evaluation_overrides', {
      evaluation_id: eval_id, trainer_id,
      field_changed: field,
      old_value: current[field],
      new_value: new_val,
      reason})
  // Apply changes + bump version + set source to 'hybrid'
  db.update('evaluations', {...changes, version: current.version+1,
              source: 'hybrid'}, eval_id)
```

7. Analytics Engine

7.1 Student-Level Analytics

Band Progression

```
func get_band_progression(student_id, limit=20):
  return db.query('''
    SELECT s.submitted_at, e.band_overall, e.band_ta, e.band_cc,
           e.band_lr, e.band_gra
    FROM submissions s
    JOIN evaluations e ON e.submission_id = s.id
    WHERE s.student_id = ? AND s.is_final = true
      AND e.status = 'approved'
    ORDER BY s.submitted_at ASC LIMIT ?
  ''', [student_id, limit])
```

Weakness Detection

```
// Weakness = criterion avg band that is > 0.5 below overall avg band
func detect_weaknesses(student_id):
  last_5 = get_last_n_evaluations(student_id, n=5)
  avgs = {ta: mean(last_5.ta), cc: mean(last_5.cc),
          lr: mean(last_5.lr), gra: mean(last_5.gra)}
  overall_avg = mean(avgs.values())
  return [c for c, v in avgs if overall_avg - v > 0.5]
```

7.2 Class-Level Analytics

```
func get_class_summary(org_id, trainer_id, date_from, date_to):
  return db.query('''
    SELECT
      AVG(e.band_ta) as avg_ta, AVG(e.band_cc) as avg_cc,
      AVG(e.band_lr) as avg_lr, AVG(e.band_gra) as avg_gra,
      AVG(e.band_overall) as avg_overall,
      COUNT(DISTINCT s.student_id) as active_students,
      COUNT(*) as total_submissions
    FROM submissions s
    JOIN evaluations e ON e.submission_id = s.id
    JOIN users u ON u.id = s.student_id
    WHERE u.org_id = ? AND s.submitted_at BETWEEN ? AND ?
  ''', [org_id, date_from, date_to])
```

Most Common Mistakes Across Class

```
func get_class_error_patterns(org_id):
  return db.query('''
    SELECT ep.pattern_type,
           COUNT(DISTINCT ep.student_id) as affected_students,
           SUM(ep.occurrence_count) as total_occurrences
    FROM error_patterns ep
    JOIN users u ON u.id = ep.student_id
    WHERE u.org_id = ?
```

```
    GROUP BY ep.pattern_type
    ORDER BY total_occurrences DESC
  ''', [org_id])
```

7.3 Predictive Analytics — Time to Target Band

Linear regression on band_overall progression per student. Extrapolates submissions-to-target assuming current improvement rate.

```
func predict_target_date(student_id):
  history = get_band_progression(student_id)
  if len(history) < 3: return null  // not enough data
  target = get_student_target_band(student_id)
  // Simple linear regression on (submission_index, band_overall)
  slope, intercept = linear_regression(
    x=[i for i in range(len(history))],
    y=[h.band_overall for h in history]
  )
  if slope <= 0: return null  // no improvement trend
  // How many more submissions to reach target?
  current_band = history[-1].band_overall
  subs_needed = (target - current_band) / slope
  // Estimate based on avg submissions per week
  avg_subs_per_week = len(history) / weeks_since_first(history)
  weeks_needed = subs_needed / avg_subs_per_week
  return today() + timedelta(weeks=weeks_needed)
```

Materialise Daily Snapshots

```
// Cron job — runs daily at 02:00 UTC
func materialise_analytics_snapshots():
  orgs = db.query('SELECT DISTINCT org_id FROM users')
  for org in orgs:
    students = get_students(org.id)
    for student in students:
      snapshot = compute_snapshot(student.id)
      db.upsert('analytics_snapshots', snapshot,
                conflict_target: [student_id, snapshot_date])
```

8. API Design

RESTful JSON API. All routes prefixed with /api/v1. Authentication via JWT (access token 15min, refresh token 7d). Org context derived from JWT claims — never passed in body.

8.1 Auth Routes

Method + Path	Request Body	Response
POST /auth/register	{name, email, password, org_slug, role}	{ user, access_token, refresh_token }
POST /auth/login	{email, password}	{ user, access_token, refresh token }
POST /auth/refresh	{refresh token}	{ access token }

8.2 Prompt Routes

Method + Path	Auth	Notes
POST /prompts/generate	trainer	AI-generates a prompt, stores it
POST /prompts	trainer	Manually create prompt
GET /prompts?task_type=&difficulty=&tags=	trainer	List + filter prompts
GET /prompts/{id}	trainer|student	Get single prompt
PATCH /prompts/{id}	trainer	Edit/archive

8.3 Assignment Routes

Method + Path	Auth	Notes
POST /assignments	trainer	Assign prompt to student or group
GET /assignments?student id=&status=	trainer	List assignments
GET /assignments/mine	student	Student's own assignments
PATCH /assignments/{id}	trainer	Update deadline, max attempts
DELETE /assignments/{id}	trainer	Soft-delete (cancel)

8.4 Submission Routes

Method + Path	Auth	Notes
POST /submissions	student	{assignment_id, body, is_final, time taken seconds}
GET /submissions/{id}	student|trainer	Get single submission with evaluation

Method + Path	Auth	Notes
GET /submissions?assignment_id=&student_id=	trainer	List submissions
PATCH /submissions/{id}	student	Update draft (is_final=false only)

8.5 Evaluation Routes

Method + Path	Auth	Notes
GET /evaluations/{id}	trainer|student	Full evaluation with feedback JSON
PATCH /evaluations/{id}	trainer	Override scores/feedback (auto-audits)
POST /evaluations/{id}/approve	trainer	Approve evaluation
GET /evaluations/{id}/history	trainer	All versions + audit trail
POST /evaluations/{id}/retry	trainer	Re-trigger AI evaluation

8.6 Analytics Routes

Method + Path	Auth	Notes
GET /analytics/student/{id}/progression	trainer|student	Band trend over time
GET /analytics/student/{id}/weaknesses	trainer|student	Weakness criteria
GET /analytics/student/{id}/prediction	trainer|student	Target date estimate
GET /analytics/class?from=&to=	trainer	Class-level aggregates
GET /analytics/class/error-patterns	trainer	Most common mistakes

9. AI Prompt Design Strategy

9.1 Prompt Engineering Principles

- Criterion isolation — one system prompt per IELTS criterion. Never score multiple criteria in one call.

- Descriptor grounding — embed the official IELTS band descriptor language for each band in the system prompt so the model has explicit criteria, not just intuition.
- JSON-only output — enforce strict JSON output with schema in system prompt. Detect and retry if output is not valid JSON.
- Few-shot examples — include one Band 5 example and one Band 7.5 example in each scoring prompt to anchor the model.
- Chain-of-thought suppressed for scoring — add 'Do not show your reasoning. Return only the JSON.' to prevent verbose CoT consuming tokens without improving accuracy.
- CoT enabled for feedback — for narrative feedback generation, allow the model to reason internally but return structured JSON only.

9.2 Token Optimisation Strategy

```
// Per-submission token budget estimate
Scoring (4 criteria x 3 runs):  4 x 3 x ~800 tokens  = ~9,600 tokens
Feedback generation:                          ~1,200 tokens
Model answer:                                   ~800 tokens
Error pattern extraction (AI pass):             ~600 tokens
──────────────────────────────────────────────────────────
Total per submission (worst case):           ~12,200 tokens
```

Cost Reduction Strategies

- Normalisation only on borderline scores: run 3x scoring calls only if run1 score is x.0 or x.5 ± 0.5 from a band boundary. Otherwise trust run1 score.
- Submission caching: SHA-256 hash of (prompt_id + submission_body). If hash exists in cache, return cached evaluation instead of re-calling LLM.
- Prompt compression: strip redundant whitespace and examples from scoring prompts after the first 1,000 calls (use A/B test to validate quality is maintained).
- Tiered model strategy: use claude-haiku for runs 2 and 3 in normalisation, claude-sonnet for run 1 and all feedback generation.

9.3 Hallucination Reduction

- Temperature=0.1 for all scoring calls — near-deterministic output.
- Explicit band scale in every prompt: 'Use ONLY these values: 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0'. Out-of-range outputs are rejected and retried.
- Schema validation: parse and validate JSON schema on return. Retry up to 3 times on schema error before failing gracefully.
- Grounding with submission text: always include the actual submission in the user prompt so the model cannot generate feedback about things that are not in the text.
- Confidence flagging: if criterion scores deviate > 1.5 bands across normalisation runs, flag evaluation as 'needs_review' for trainer attention.

10. Multi-Tenant Architecture

10.1 Tenancy Model

Logical multi-tenancy via org_id column on all tenant-scoped tables. No separate schemas per org (avoids migration complexity at scale). Row-Level Security (RLS) policies enforced at DB level as a second line of defence after application-level filtering.

```
// PostgreSQL RLS example
ALTER TABLE submissions ENABLE ROW LEVEL SECURITY;
CREATE POLICY org_isolation ON submissions
  USING (org_id = current_setting('app.current_org_id')::uuid);
```

10.2 RBAC (Role-Based Access Control)

Role	Can Do	Cannot Do
admin	Manage org, invite users, billing, all trainer actions	Nothing
trainer	Create prompts, assign tasks, view all student data, override evaluations	Manage billing, delete org
student	View own submissions, own evaluations, own analytics	View other students, override evaluations

10.3 Middleware Enforcement

```
func auth_middleware(request):
  token = extract_bearer_token(request)
  claims = verify_jwt(token)  // {user_id, org_id, role}
  // Inject org context for RLS
  db.execute('SET app.current_org_id = ?', claims.org_id)
  request.user = claims
  next()
```

11. Scalability Considerations

11.1 Evaluation Queue

LLM calls are async. Submissions trigger a job queue event, not a synchronous HTTP call. Student sees 'Evaluation in progress' state, polls for completion or receives webhook/websocket update.

```
// On submission.finalised event:
queue.add('evaluate_submission', { submission_id }, {
  attempts: 3,
  backoff: { type: 'exponential', delay: 2000 }
})
```

11.2 Caching Strategy

Cache Key	TTL	What's Cached
`eval:{sha256(prompt+submission)}`	30 days	Full evaluation JSON for identical inputs
`student_analytics:{student_id}`	24 hours	Progression + weaknesses
`class_summary:{org_id}:{date}`	6 hours	Class-level aggregates
`prompt list:{org id}:{filters hash}`	5 minutes	Prompt listing pages

11.3 Database Indexes

```
CREATE INDEX idx_submissions_student ON submissions(student_id, submitted_at DESC);
CREATE INDEX idx_submissions_assignment ON submissions(assignment_id, is_final);
CREATE INDEX idx_evaluations_submission ON evaluations(submission_id, version DESC);
CREATE INDEX idx_error_patterns_student ON error_patterns(student_id, pattern_type);
CREATE INDEX idx_analytics_snapshots ON analytics_snapshots(student_id, snapshot_date);
```

11.4 Rate Limiting

- LLM API calls: max 10 concurrent evaluations per org on free plan, 50 on pro, unlimited on enterprise.
- Submission endpoint: max 5 submissions per student per hour (prevents abuse).
- Prompt generation: max 20 AI-generated prompts per day per org on free plan.

Final Note

This architecture is designed to be built incrementally. Start with: auth + submissions + single-criterion scoring (Stage 2) + basic feedback. Ship that, validate with real students, then layer in normalisation, analytics, and trainer overrides. Every module is independently testable and deployable.